

Tema 8: Comunicación TLS

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

Certificados X.509

Campos X.509

- ▶ 1. Certificate
 - ▶ (a) Version Number
 - ▶ (b) Serial Number
 - ▶ (c) Signature Algorithm ID
 - ▶ (d) Issuer Name
 - ▶ (e) Validity Period
 - ▶ i. Not Before
 - ▶ ii. Not After
 - ▶ (f) Subject Name
 - ▶ (g) Subject Public Key Info
 - ▶ i. Public Key Algorithm
 - ▶ ii. Subject Public Key
 - ▶ (h) Issuer Unique Identifier (optional)
 - ▶ (i) Subject Unique Identifier (optional)
 - ▶ (j) Extensions (optional)
- ▶ 2. Certificate Signature Algorithm
- ▶ 3. Certificate Signature

- ▶ La adición más importante de la versión 3 es el apartado de extensiones, que permite agregar más comprobaciones de seguridad
- ▶ La versión 1 y 2 se siguen utilizando y se consideran también seguros
- ▶ Recuerden que el propósito principal de un certificado es ligar la identidad de un subject a una llave pública bajo la firma de un issuer (entidad de confianza)
- ▶ Los campos que identifican al subject, la llave pública y el issuer son los más críticos, los demás campos proveen de contexto necesario para entender e interpretar los datos

1. *Journal of Management Studies*, 1996, 33(1), 1-14.

Campos X.509

- ▶ Por ejemplo, un issuer o subject puede ser representado de esta forma:
- ▶ CN=Charlie, OU=Espionage, O=EA, L=Room 110, S=HQ, C=EA
- ▶ No todos los sub-campos son obligatorios pero CN es usualmente el campo crítico
- ▶ El CN es el identificador principal en la validación de un certificado
- ▶ En certificados modernos (versión 3) también se suele incluir un campo adicional: Subject Alternative Name
- ▶ En realidad el CN suele ser un nombre de dominio como google.com

Campos X.509

- ▶ Una vez identificados el issuer y el subject, los campos restantes son la llave pública del subject y la firma del contenido del certificado
- ▶ La firma se calcula sobre el binario del certificado bajo una codificación llamada DER (Distinguished Encoding Rules)
- ▶ La firma prueba tanto que el certificado no ha sido modificado como que fue aprobado por el issuer verdadero

- ▶ Se requieren para crear certificados
- ▶ Una entidad interesada en tener un certificado, crea un CSR (Certificate Signing Request) y lo manda a un CA (Certificate Authority)
- ▶ Un CSR tiene casi los mismos campos que un certificado X.509 pero no contiene los campos referentes al issuer, dado que esos campos los debe establecer el CA
- ▶ Una vez el CA tiene el CSR, usa su propio certificado y llave privada asociada para generar el certificado con los campos faltantes

- ▶ El campo issuer de un certificado debe ser igual al campo subject del certificado de quien firma
- ▶ Una vez todos los campos son rellenados el CA firma el certificado con su llave privada
- ▶ Obviamente la entidad que manda el CSR nunca manda su llave privada al CA, el CSR sólo tiene su llave pública, nadie, ni siquiera el CA debe tener la llave privada

- ▶ Como ya se mencionó antes, los certificados X.509 son usualmente codificados en un formato binario llamado DER
- ▶ Sin embargo, la mayoría de representaciones en disco de certificados utiliza el formato PEM (visto en temas anteriores), también se utiliza PEM para llaves privadas y CSR
- ▶ PEM es un formato de texto, lo que facilita su transferencia a través de protocolos basados en texto como los propios del correo electrónico

Generar una llave privada

- ▶ En este caso se usa una llave RSA
- ▶ Esto lo hemos hecho muchas veces con python, pero no con openssl
- ▶ `openssl genpkey -algorithm RSA -out domain_key.pem -pkeyopt rsa_keygen_bits:2048`
- ▶ Se debe tener cuidado con estos comandos y los "tutoriales en internet", muchas veces se recomienda usar el comando `genrsa`, se debe preferir usar siempre `genpkey` que es más general
- ▶ También es muy importante, como ya se ha visto en el curso, que el tamaño de la llave sea de al menos 2048 bits

Generar CSR a través de una llave

- ▶ El proceso de generación de CSR extrae la llave pública de la llave privada y la agrega a la petición
- ▶ `openssl req -new -key domain_key.pem -out domain_request.csr`
- ▶ El comando anterior es interactivo, pero el único dato obligatorio para TLS es el referente al nombre de subject, sin embargo, algunos CA requieren de los demás campos
- ▶ El certificado se genera en formato PEM, si se desea ver todo de una forma legible para los humanos:
- ▶ `openssl req -in domain_request.csr -text`

Generar CSR a través de una llave

- ▶ La versión generada es la 1, y no la 3, OpenSSL siempre asigna esta versión a menos que se usen los campos de extensión
- ▶ En el certificado final, el issuer podría agregar campos de extensión para tener un certificado versión 3
- ▶ Otros campos como "Serial Number" no están presentes, ya que son agregados por el CA
- ▶ Otra cosa interesante es que el CSR está firmado, en este caso por la llave privada del subject, para demostrar que posee dicha llave
- ▶ Cualquiera podría agregar cualquier llave pública a un CSR si no se tuviera la comprobación de firma, a esta comprobación se le llama **prueba de posesión**

Firmar un CSR para producir un certificado

- ▶ Recordemos que los certificados necesitan estar firmados por el CA, para que el subject pueda establecer la validez de su identidad
- ▶ El CA es responsable de cierto nivel de verificación
- ▶ Si alguien trata de reclamar una identidad que no posee, el CA debería realizar un proceso de validación y denegar la petición
- ▶ Un atacante (y cualquiera en realidad) tiene también la posibilidad de crear un certificado auto-firmado, osea firmado con la misma llave privada del subject
- ▶ Recordemos que los certificados auto-firmados son utilizados por los certificados raíz de los CA raíz, ya que la cadena de dependencias de CAs debe terminar en algún punto
- ▶ En un certificado auto-firmado el subject e issuer son iguales

100

- ▶ openssl permite generar certificados auto-firmados
- ▶ Estos certificados pueden ser usados para testing, por ejemplo para probar sistemas web bajo https
- ▶ También pueden ser mal utilizados por atacantes
- ▶ `openssl x509 -req -days 30 -in domain_request.csr -signkey domain_key.pem -out domain_cert.crt`
- ▶ Para ver el contenido: `openssl x509 -in domain_cert.crt -text`

- ▶ Una vez se tiene un certificado, se puede utilizar para firmar certificados nuevos
- ▶ El siguiente ejemplo muestra cómo hacerlo, usando para ejemplificar llaves EC
- ▶ Primero se debe generar el CSR, y para eso se necesita un par de llaves:
- ▶ `openssl genpkey -algorithm EC -out localhost_key.pem -pkeyopt ec_paramgen_curve:P-256`
- ▶ En este caso se usó la curva P-256 de NIST, otra opción popular es 25519 pero no es soportada por cryptography

- ▶ Se genera ahora el CSR para el CN 127.0.0.1 (más adelante se verá porqué)
- ▶ `openssl req -new -key localhost_key.pem -out localhost_request.csr`
- ▶ Con el certificado auto-firmado generado antes, es posible generar un certificado asociado al CSR, para esto se necesita la llave privada del CA (la llave privada asociada a la llave pública del certificado auto-firmado)
- ▶ En el ejemplo se utilizarán algunas opciones de la versión 3 para limitar el uso del certificado generado
- ▶ En este caso no se permitirá que se puedan crear certificados nuevos con el certificado generado

- ▶ La limitación introducida es para evitar abuso si es que de alguna forma un atacante logra robar o engañar a un CA para tener un certificado con su llave privada correspondiente
- ▶ Lo anterior refuerza la cadena de autoridad de los certificados, no cualquiera debería poder ser un CA
- ▶ También es útil limitar el uso que se le puede dar a la llave pública contenida en el certificado, para que por ejemplo sólo se pueda usar para firmas digitales (otro uso puede ser firmar listas de revocación de certificados CRLs)

- ▶ Para agregar las opciones de la versión 3, se puede crear un archivo de texto, en este caso llamado v3.ext, que contiene dos líneas:
 - ▶ `keyUsage=digitalSignature`
`basicConstraints=CA:FALSE`
- ▶ Para firmar el CSR:
 - ▶ `openssl x509 -req -days 365 -in localhost_request.csr -CAkey domain_key.pem -CA domain_cert.crt -out localhost_cert.crt -set_serial 123456789 -extfile v3.ext`

Attackers might be trying to steal your information from 127.0.0.1 (for example, passwords, messages, or credit cards). [Learn more](#)

☐ Help improve Safe Browsing by sending some system information and page content to Google.
[Privacy policy](#)

Back to safety

100

- ▶ Si se activan las opciones de desarrollador, es posible ver más información del problema
- ▶ El problema es que el certificado raíz no es parte de los certificados raíz de confianza del navegador
- ▶ Ojo: esto no quiere decir que los datos en la conexión viajen sin protección criptográfica, el problema de seguridad es un posible spoofing en los certificados

Confianza en CA

- ▶ Pero, ¿Cómo es que los navegadores confían en alguna organización?
- ▶ La respuesta desgraciadamente es más arbitraria de lo que debería
- ▶ Un número pequeño de "autoridades" se han establecido a si mismas como autoridades raíz confiables
- ▶ Estas organizaciones y compañías tienen sus certificados raíz instalados por defecto en SOs populares y en navegadores web
- ▶ Toda la demás confianza se deriva de esas autoridades arbitrarias

100

- ▶ Realiza una investigación sobre las autoridades de confianza raíz en Internet, cubriendo los siguientes puntos:
- ▶ ¿Aproximadamente cuántas son y como se distribuyen?
- ▶ ¿Cómo se puede llegar a ser una entidad de este tipo?
- ▶ Implicaciones de seguridad

- ▶ Necesario para el uso de HTTPS
- ▶ Si deseas tener un certificado para tu sitio en internet (prácticamente obligatorio hoy en día) es usualmente necesario probar que tienes control sobre el dominio del sitio
- ▶ Esto se logra con protocolos como ACME (ver documentación de let's encrypt)
- ▶ También es común contar con este servicio a través de un webhosting, aunque suele ser de pago

Journal of Management Education 36(7) 809–826

TLS 1.2 y 1.3

- ▶ TLS inicia con un handshake, el cual es en extremo crítico
- ▶ TLS tiene dos objetivos principales:
 - ▶ Establecer identidad
 - ▶ Derivar llaves de sesión para el transporte seguro
- ▶ Estos dos objetivos son logrados típicamente mediante el handshake

- ▶ También es en el handshake donde se encuentran las diferencias más importantes entre la versión 1.2 y 1.3
- ▶ Primero se revisará el handshake de la versión 1.2 y luego de la versión 1.3

[illegible]

100



- ▶ La configuración más popular de TLS actualmente autentica sólo al servidor cuando el certificado del servidor se envía con el ServerHello
- ▶ A menos que el servidor lo especifique de forma explícita, el cliente no envía un certificado para autenticarse
- ▶ Para muchas de las aplicaciones en Internet, esto es suficiente
- ▶ Cuando ambas partes se autentican con un certificado, se le llama MTLS (Mutual TLS)

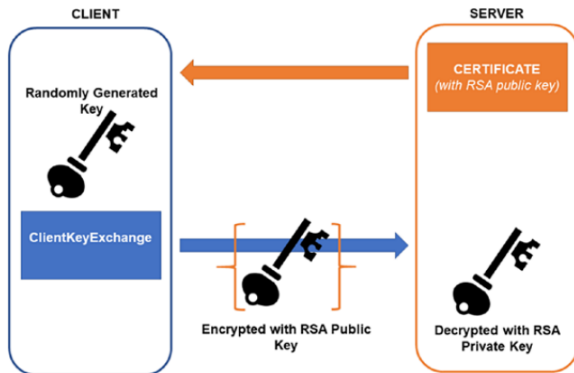
Autenticación de cliente

- ▶ En Internet los clientes suelen ser libres de entrar a los servicios que quieran
- ▶ Los servidores deben asegurarle a los clientes que son el servicio verdadero que espera el cliente
- ▶ La identidad del cliente es algo menos claro
- ▶ La identidad de un servidor está usualmente ligada a un dominio o dirección IP
- ▶ Lo anterior no es posible hacerlo con un cliente, además el cliente es una entidad más dinámica que puede utilizar una variedad de dispositivos y medios de comunicación

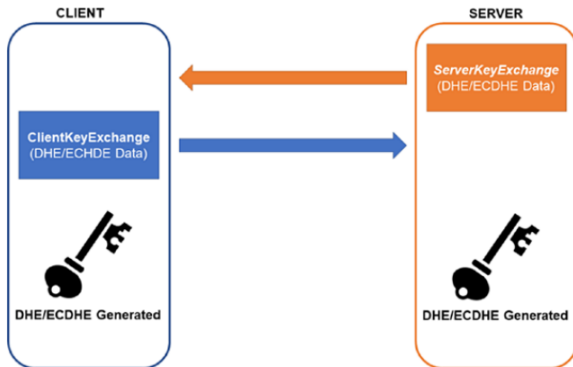
- ▶ Para circunstancias donde se necesita autenticar al cliente, como en una transacción bancaria, lo que importa es la identidad del usuario, más que la identidad de la máquina
- ▶ Para este fin se utilizan mecanismos como los clásicos usuario-contraseña u otros a nivel aplicación
- ▶ Después de haber autenticado al servidor y creado una sesión segura con una llave compartida, el usuario puede enviar de forma segura sus datos de identidad

- ▶ Notar que con transporte de llaves RSA no se necesita algoritmo de firma digital, pues no hay información que se necesite autenticar
- ▶ En este tipo de transporte el cliente recibe el certificado del servidor que fue enviado con el hello del servidor, extrae la llave pública y cifra el PMS con dicha llave para enviarla al servidor, sólo el servidor podrá descifrar el PMS (por esto no se requiere de firmas)
- ▶ De esta forma tanto cliente como servidor comparten el mismo PMS

Derivación de llaves de sesión

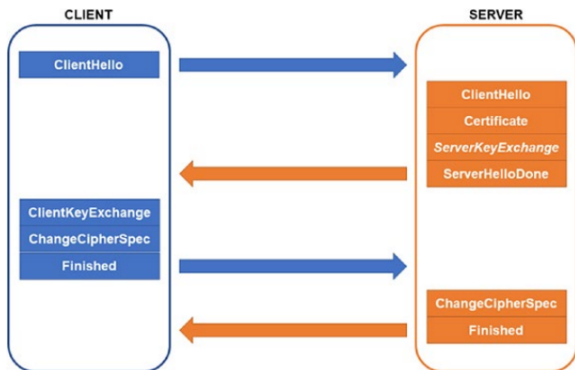


Derivación de llaves de sesión



- ▶ Por ejemplo, un atacante podría haber alterado el mensaje de hello del cliente para dejar sólo cipher suits débiles
- ▶ Tanto cliente como servidor llevan un registro de todos los mensajes enviados, así la otra parte puede comprobar integridad con el hash de los mensajes recibidos contra el hash del mensaje Finished
- ▶ De no pasarse la comprobación se considera que el canal está comprometido y se cierra de inmediato

Cambio a cifrado



- ▶ Después del handshake, el cliente ya verificó la identidad del servidor y ambas partes comparten un PMS
- ▶ Con el PMS y datos adicionales (como los nonces de cliente y servidor) derivan llaves simétricas
- ▶ Estas llaves simétricas tienen el objetivo de brindar confidencialidad, integridad y autenticación en cada mensaje

Derivación de llaves y transferencia en masa

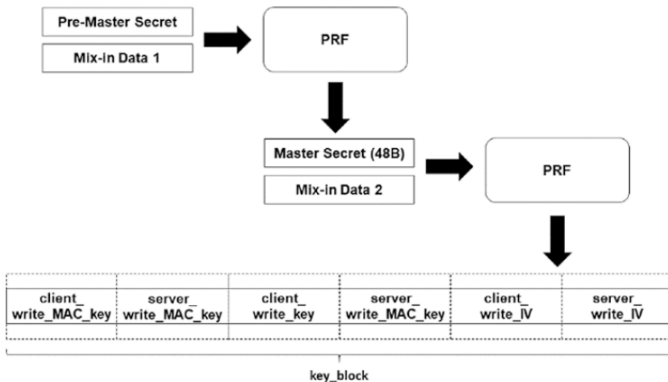
- ▶ En el caso de TLS el PMS se expande (cual sea su longitud) a 48 bytes para generar el master secret
- ▶ El master secret es a su vez expandido a tantos bytes sea necesario para obtener todas las llaves de sesión e IVs requeridos por el cipher suite
- ▶ Con todas las expansiones del master secret se obtiene el key_block que varía en tamaño de acuerdo al cipher suite

Derivación de llaves y transferencia en masa

El `key_block` tiene a lo mucho 6 parámetros:

- ▶ client write MAC key
- ▶ server write MAC key
- ▶ client write key
- ▶ server write key
- ▶ client write IV
- ▶ server write IV

Derivación de llaves y transferencia en masa



- ▶ No todos los cipher suits usan todos los campos, por ejemplo, si se utiliza AES-GCM los campos de MAC no son necesarios
- ▶ Sin embargo, todos los cipher suits proveen integridad, confidencialidad y autenticación
- ▶ En el pasado se han encontrado vulnerabilidades en algunos cipher suites, por ejemplo en TLS 1.0 AES-CBC tenía una vulnerabilidad de ataque de padding oráculo derivado de aplicar MAC y luego cifrar (recuerden que lo correcto es cifrar luego MAC)

Derivación de claves y transferencia en masa

```
struct {
    ContentType type;
    ProtocolVersion version;
    uint16 length;
    select (SecurityParameters.cipher_type) {
        case stream: GenericStreamCipher;
        case block: GenericBlockCipher;
        case aead: GenericAEADCipher;
    } fragment;
} TLSCiphertext;
```

Derivación de llaves y transferencia en masa

```
stream-ciphered struct {
    opaque content[TLSCompressed.length];
    opaque MAC[SecurityParameters.mac_length];
} GenericStreamCipher ;

struct {
    opaque IV[SecurityParameters.record_iv_length];
    block-ciphered struct {
        opaque content[TLSCompressed.length];
        opaque MAC[SecurityParameters.mac_length];
        uint8 padding[GenericBlockCipher.padding_length];
        uint8 padding_length;
    };
} GenericBlockCipher;
```

Derivación de claves y transferencia en masa

```
struct {  
    opaque nonce_explicit[SecurityParameters.record_iv_length];  
    aead-ciphered struct {  
        opaque content[TLSCompressed.length];  
    };  
} GenericAEADCipher ;
```

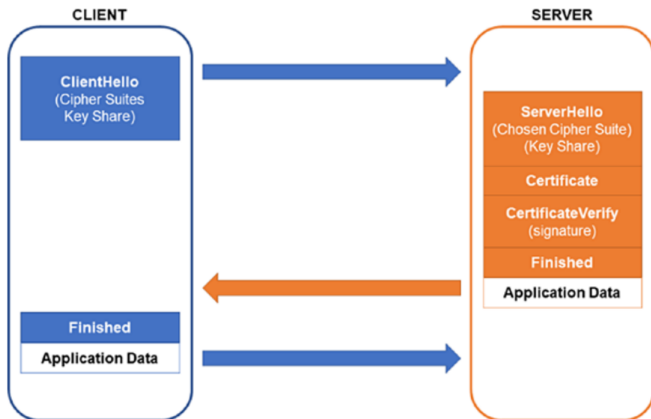
TLS 1.3

- ▶ Esta versión hace grandes cambios al proceso de handshake
- ▶ TLS 1.3 desecha la mayoría de cipher suits presentes en TLS 1.2, permitiendo sólo 5, todos basados en AEAD:
 - ▶ TLS_AES_256_GCM_SHA384
 - ▶ TLS_CHACHA20_POLY1305_SHA256
 - ▶ TLS_AES_128_GCM_SHA256
 - ▶ TLS_AES_128_CCM_8_SHA256
 - ▶ TLS_AES_128_CCM_SHA256

TLS 1.3

- ▶ Básicamente se da soporte a AES-GCM, AES-CCM y a ChaCha20-Poly1305
- ▶ Con estas reducciones se vuelve más difícil equivocarse en la configuración permitiendo chipers débiles
- ▶ También se vuelve más difícil para los atacantes engañar para que se use un cipher suite débil (ya no hay ninguno débil)
- ▶ RSA ya no se puede utilizar como transporte de llaves, sólo DHE y ECDHE
- ▶ Un cambio importante en el handshake es que ahora todo el proceso se hace en una sola ronda de mensajes

TLS 1.3



100

TLS 1.3

- ▶ Al no haber transporte de llaves RSA, la confidencialidad hacia adelante se vuelve obligatoria
- ▶ Limitar a sólo AEAD es también una mejora importante
- ▶ **CUIDADO:** existe una "variante" de TLS 1.3 conocida como eTLS, la cual no es oficial y remueve mucha de la seguridad importante de TLS, como la confidencialidad hacia adelante
- ▶ Supuestamente esta variante es más eficiente y fácil de usar, pero debilita la seguridad, nunca debes de utilizarla, ni deberías utilizar navegadores web que le dan soporte, en el futuro a esta tecnología pretenden llamarla Enterprise Transport Security (ETS)

- ▶ En el certificado del cliente, el nombre de subject debe corresponder al nombre del host (dominio) en la URL:
 - ▶ El nombre de host puede corresponderse con el CN del subject
 - ▶ El nombre de host puede corresponderse con el nombre alternativo del subject (en una extensión V3)
- ▶ Ninguno de los certificados en la cadena puede estar expirado
- ▶ Ninguno de los certificados en la cadena puede estar revocado
- ▶ El issuer de un certificado debe ser el subject del certificado en el siguiente elemento de la cadena, hasta llegar a la raíz
- ▶ Limitaciones de certificado (como KeyUsage y BasicConstraints) son reforzados
- ▶ Políticas adicionales son reforzadas, como longitud máxima de la cadena, nombrado, etc.

Revocación de certificados: CRLs

- ▶ Registro estático de certificados que han sido revocados (mediante su identificador)
- ▶ Los CRLs son a menudo específicos para un CA y son firmados por el CA
- ▶ Es importante que el CA lleve un registro de los certificados generados y que no repita identificadores
- ▶ Los CRLs se suelen publicar de forma periódica (una vez al día por ejemplo)
- ▶ Los sistemas de verificación de certificados, como los usados en TLS, deben mantener una lista de todos los certificados revocados, para poder detectar e invalidar certificados revocados

- ▶ Se utiliza para revisar la validez de los certificados a partir de su número de serie
- ▶ A diferencia de CRLs, el protocolo se utiliza con un servidor online en tiempo real, al cual se le puede pedir que revise la revocación en el momento que se hace la verificación
- ▶ Usualmente el CA tiene el control de su propio servidor OCSP para la verificación de sus certificados generados

Revocación de certificados: OCSP

- ▶ Obviamente OCSP tiene información más actualizada que los CRLs
- ▶ Pero introduce latencia adicional en el handshake de TLS
- ▶ Además introduce otro problema: ¿Qué debe hacer el cliente si el servidor OCSP no responde?
- ▶ La mayoría de navegadores asumen que el certificado no ha sido revocado
- ▶ Lo anterior es una posible explotación, que aproveche un ataque DOS al servidor OCSP

Revocación de certificados

- ▶ Por los problemas antes mencionados, CRLs y OCSP se consideran en la actualidad obsoletos
- ▶ Algunos navegadores como Chrome o Firefox ni siquiera utilizan o dejan utilizar estos mecanismos, sino que mantienen sus propias listas
- ▶ La revocación de certificados es todavía un problema abierto
- ▶ En la actualidad se exploran nuevos métodos como OCSP stapling

Raíces no confiables

- ▶ Un problema fundamental de TLS es que la seguridad depende de tener entidades de confianza
- ▶ Pero como dijo el poeta Romano Juvenal: "¿Quis custodiet ipsos custodes?" (¿Quién custodia a los custodios?)
- ▶ Un problema grave es que si se compromete la llave privada de un CA, un atacante puede generar certificados nuevos para cualquier dominio, haciendo que los clientes confíen en dicho certificado

Raíces no confiables

- ▶ El anterior no es un problema teórico, ha sucedido en el pasado
- ▶ Por ejemplo, en 2011 atacantes lograron robar una llave privada del CA DigiNotar, generando certificados con subdominios aparentes de Google, Yahoo, Wordpress, Mozilla y TOR
- ▶ El CA tuvo que ser removido de los CAs de confianza de todos los navegadores y OS, lo que provocó también que la organización cerrara
- ▶ En realidad nunca se supo el alcance completo del ataque

Raíces no confiables

- ▶ No sólo un CA puede ser atacado y comprometido, sino que algunos CAs llevan a cabo prácticas dudosas
- ▶ Por ejemplo, la empresa Trustico que se dedica a revender certificados (genera los certificados a nombre de otros) le pidió a un CA DigiCert que revocara más de 20 mil certificados, esto debido a que se perdió la confianza en dichos certificados
- ▶ Lo que pasó es que Trustico guardaba una copia de todas las llaves privadas de los certificados que revendía, lo cual de por sí es una práctica deshonesta que puede provocar que un empleado robe las llaves o que se comprometan de otra forma
- ▶ Además Trustico en algún momento envió las llaves privadas de los 20 mil certificados a través de email, razón por la cual se pidió la revocación

Raíces no confiables

- ▶ Un cliente nunca debe confiarle a un CA su llave privada, de hacerlo siempre corre un riesgo
- ▶ Toda la seguridad criptográfica depende de que los secretos se mantengan secretos
- ▶ El problema de los certificados fraudulentos o usados de forma incorrecta es muy común y serio
- ▶ Existen algunos mecanismos para mitigar estos problemas, los cuales se discuten a continuación

Pinning de certificados

- ▶ La idea básica es que el cliente, antes de recibir el certificado, ya tiene una expectativa de lo que el certificado debe tener
- ▶ Cuando se recibe el certificado se compara contra la versión esperada (versión pinned)
- ▶ Y se aplican políticas cuando se encuentran discrepancias, bajo el supuesto de que esas discrepancias representan un posible ataque
- ▶ Esta aproximación también se le suele llamar HTTP Public Key Pinning (HPKP)

Pinning de certificados

- ▶ En algún punto la tecnología fue apoyada por Google, pero en la actualidad se considera insuficiente
- ▶ Sin embargo, se sigue utilizando en ámbitos como las aplicaciones móviles, donde el certificado del autor de la app se agrega al empaquetado de la propia app, este certificado pinned, siempre se compara con el certificado recibido en una comunicación TLS que hace la app
- ▶ Si se necesita un nuevo certificado, se saca una nueva versión de la app
- ▶ Google y Firefox hacen algo similar en sus navegadores, por ejemplo, Google notó el problema con DigiNotar mediante este mecanismo

Pinning de certificados

- ▶ HPKP utiliza el principio TOFU (trust on first use), donde básicamente la primera vez que visitas un sitio el cliente hace un pinning del certificado asociado y lo mantiene un tiempo
- ▶ HPKP es interesante pero introduce muchos problemas adicionales y puede tener explotaciones, por lo que ha ido quedando en desuso

Transparencia de certificados

- ▶ Este es un mecanismo que está ganando fuerza en los últimos tiempos
- ▶ La idea es similar a blockchain, donde cada vez que se genera un certificado se agrega también a un log público
- ▶ El log lo mantiene un tercero, pero como con blockchain, no es necesario confiar en el tercero porque todo lo generado está fuertemente atado por verificaciones de hash

Transparencia de certificados

- ▶ El propósito del log es transparencia: los CAs pueden ser auditados por los certificados que producen
- ▶ La meta es tener todos los certificados generados de forma pública, para que puedan ser criptográficamente verificados
- ▶ Es probable que pronto los navegadores sólo confíen en certificados que aparecen en el log

Transparencia de certificados

- ▶ Esto limitará mucho a los atacantes, pues certificados falsos tendrán que hacerse públicos, lo que les permitirá a los CAs detectarlos y revocarlos rápidamente de forma automática
- ▶ Incluso si un atacante logra salirse con la suya, se tendrá un registro público de todo lo que ha hecho con los certificados, pudiendo establecer el alcance y daño del ataque
- ▶ En el caso de DigiNotar, el hecho de no conocer cuántos certificados falsos se habían generado, provocó que se tuviera que eliminar DigiNotar por completo
- ▶ Esta tecnología es todavía nueva y está evolucionando, no cuenta aun por ejemplo con mecanismos de revocación, pero es muy probable que en el futuro sea el estándar a seguir

Ataques conocidos a TLS

Introducción

- ▶ Un atacante tiene varias opciones además de los que el manejo de certificados presentan
- ▶ En la actualidad existen otros tipos de ataques con los que hay que tener cuidado
- ▶ A continuación se mencionarán varios ataques y la forma de prevenirlos

POODLE

- ▶ Padding Oracle On Downgraded Legacy Encryption
- ▶ TLS 1.0 puede ser explotado cuando se usa CBC ya sea con AES o con un cifrador más antiguo llamado DES
- ▶ TLS 1.1 y 1.2 supuestamente arregla el problema cambiando la forma en que se hace el padding
- ▶ Sin embargo, muchos servidores pueden ser atacados para re-negociar el protocolo a 1.0

POODLE

- ▶ Además, se descubrió que algunas implementaciones de TLS 1.1 y 1.2 usaban el mismo padding de TLS 1.0
- ▶ Las defensas incluyen:
 - ▶ Deshabilitar TLS 1.0 y 1.1
 - ▶ Verificar servicio TLS 1.2 mediante una herramienta de auditoría (scanner de vulnerabilidad por ejemplo)
 - ▶ Migrar a TLS 1.3 y sólo permitir esta versión ya que no soporta CBC

FREAK y Logjam

- ▶ Logjam depende de forzar al servidor a bajar de versión, en este caso para tener versiones débiles de cipher suites
- ▶ En los 90's el gobierno de EU tenía una política de no permitir que se exportara criptografía fuerte a otros países, el gobierno trataba a los algoritmos fuertes como si fueran armas
- ▶ La seguridad informática todavía lleva cicatrices debido a esa política

FREAK y Logjam

- ▶ A partir de la política surgieron cipher suites específicos para TLS, conocidos como algoritmos EXPORT, los cuales eran muy débiles
- ▶ En Logjam el hello del cliente es interceptado, cambiando la lista de cipher suits a versiones EXPORT de Diffie-Hellman, con llaves fácilmente rompibles
- ▶ Con el hash del mensaje Finished debería notarse esta manipulación, sin embargo, como el Finished va cifrado de forma débil, un atacante puede interceptarlo y cambiarlo

FREAK y Logjam

- ▶ FREAK es muy parecido a Logjam, pero en vez de atacar DH ataca el transporte de llaves RSA a partir de versiones EXPORT
- ▶ Defensas contra ambos ataques incluyen:
 - ▶ Deshabilitar cipher suits débiles, especialmente versiones EXPORT
 - ▶ Usar clientes que se reusen a aceptar parámetros débiles de DH/ECDH o RSA

Sweet32

- ▶ Está pensado específicamente para cifradores de bloque que usan bloques de 64 bits
- ▶ En la mayoría de instalaciones de TLS 1.2 sólo hay un cifrador con esas características: 3DES
- ▶ 3DES utiliza por debajo DES, un cifrador simétrico con debilidades
- ▶ El tamaño de bloque impacta en que tantos datos se pueden cifrar con una misma llave antes de que se empiecen a repetir parámetros en diferentes bloques
- ▶ Para bloques de 64 bits el límite es 32 GB, lo cual es fácilmente generado por computadoras modernas

Sweet32

- ▶ Muchas implementaciones de TLS no refuerzan un límite máximo de datos cifrados con la misma llave
- ▶ Sweet32 aprovecha lo anterior para mandar suficientes datos para forzar la aparición de colisiones y recuperar datos
- ▶ Las defensas incluyen:
 - ▶ Deshabilitar cualquier cipher suite que utilice 3DES, o cualquier otro que use bloques de 64 bits

ROBOT

- ▶ Return Of Bleichenbacher's Oracle Threat
- ▶ Este ataque aprovecha los defectos del padding PKCS 1.5 para RSA, para realizar un ataque de padding oráculo
- ▶ Todas las versiones de TLS hasta la 1.2 tienen esta versión de padding, a pesar de que los defectos se descubrieron desde 1999
- ▶ Los diseñadores de TLS decidieron dejar este padding por razones de compatibilidad, y trataron de reforzarlo de otras formas

100

- ▶ En teoría lo que basta es que el servidor le deje de servir de oráculo al atacante, para no revelar información sobre el padding
- ▶ Sin embargo, estas medidas no siempre funcionan, y se descubrieron otras formas para que el servidor dé información de padding
- ▶ Defensas incluyen:
 - ▶ Deshabilitar todos los cipher suites que usen RSA para intercambio de llaves

CRIME, TIME y BREACH

- ▶ TLS 1.2 permite comprimir datos antes de cifrar, esto fue deshabilitado en 1.3
- ▶ El problema con la compresión es que revela información a los atacantes, la cual puede ser utilizada para recuperar información cifrada
- ▶ CRIME (Compression Ratio Info-leak Made Easy), se demostró primero en 2012
- ▶ La compresión sólo funciona si hay datos repetidos, si tu sólo tienes un texto cifrado que contiene un texto comprimido y si puedes insertar o parcialmente insertar mensajes para ver el efecto de la compresión y notas que la compresión a mejorado con la inserción del mensaje, eso quiere decir que agregaste algo repetido en el mensaje original

- ▶ Esta información puede ser utilizada para recuperar un número pequeño de bytes
- ▶ Cualquier pérdida de confidencialidad, por pequeña que sea, es inaceptable
- ▶ Por ejemplo, los bytes pueden representar una cookie web con información de autenticación
- ▶ CRIME fue seguido por TIME, el cual es más efectivo
- ▶ TIME inspiró a BREACH, que es un ataque diferente pero que también ataca a la compresión para revelar información
- ▶ Defensas incluyen:
 - ▶ Deshabilitar compresión

Heartbleed

- ▶ Esta no es propiamente una vulnerabilidad de TLS, sino un bug en la implementación de OpenSSL
- ▶ Este bug se daba en una extensión que permitía detectar conexiones muertas a partir de heartbeats
- ▶ Un heartbeat es un mensaje que se manda de forma periódica para que un servicio sepa que un cliente sigue activo
- ▶ Una petición típica de heartbeat incluye algunos datos que se mandan de ida y vuelta (eco) con su correspondiente longitud

Heartbleed

- ▶ Con el bug no se revisaba que la información devuelta fuera correcta
- ▶ Si la longitud del mensaje excedía el tamaño real, la implementación leía contenido de la memoria
- ▶ Aunque no hubiera garantías de que se podía leer de la memoria, podría incluir por ejemplo una llave cifrada u otros secretos
- ▶ Defensas incluyen:
 - ▶ Mantener las bibliotecas y aplicaciones actualizadas

Sockets TLS con python

- ▶ A continuación se muestra el uso de la biblioteca ssl de python, la cual tiene soporte para TLS a través de OpenSSL
- ▶ Sólo se presentará la creación de sockets cliente y servidor, los procesos de generación de certificados pueden realizarse con OpenSSL directamente